

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Import

```
In [ ]: # Import
import os
import warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import random
from keras.callbacks import EarlyStopping
from tensorflow.keras import regularizers
from tensorflow.keras.layers import Dropout

os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
warnings.filterwarnings("ignore", message="Found untraced functions such as _update_ste

# quicker running
os.environ['N_JOBS'] = '-1'

# files
# check the feature weight list from PART2-feature selection
data_train = pd.read_csv("data_train.csv")
data_val = pd.read_csv("data_val.csv")
data_test = pd.read_csv("data_test.csv")
features_list = pd.read_csv("features_list_v2.csv")

data_train.drop(columns='Unnamed: 0', inplace=True)
data_val.drop(columns='Unnamed: 0', inplace=True)
data_test.drop(columns='Unnamed: 0', inplace=True)
features_list.drop(columns='Unnamed: 0', inplace=True)
```

Get X and Y

```
In [ ]: def set_seed(seed_value):
    os.environ['TF_DETERMINISTIC_OPS'] = '1'
    os.environ['PYTHONHASHSEED']=str(seed_value)
    random.seed(seed_value)
    np.random.seed(seed_value)
    tf.random.set_seed(seed_value)

def mydic(ht,df):
    df.loc[len(df)] = ht
    return df

def get_X_Y_by_features_list(d, f):
    d_X = d[f]
    d_Y = d['Y']
    # print(d_X.shape, d_Y.shape)
    return d_X,d_Y

def get_X_Y_all():
    fs = features_list
    f = fs['feature name'].to_list()

    train_X,train_Y = get_X_Y_by_features_list(d=data_train, f=f)
    val_X,val_Y = get_X_Y_by_features_list(d=data_val, f=f)
    test_X,test_Y = get_X_Y_by_features_list(d=data_test, f=f)

    return train_X,train_Y, val_X,val_Y, test_X,test_Y

def get_X_Y_all_n(best_fs=29):
```

```

fs = features_list
f = fs['feature name'].to_list()[:]
f = f[:best_fs]

train_X, train_Y = get_X_Y_by_features_list(d=data_train, f=f)
val_X, val_Y = get_X_Y_by_features_list(d=data_val, f=f)
test_X, test_Y = get_X_Y_by_features_list(d=data_test, f=f)

return train_X, train_Y, val_X, val_Y, test_X, test_Y

set_seed(5240)
train_X, train_Y, val_X, val_Y, test_X, test_Y = get_X_Y_all()

```

```

In [ ]: print("in data_train", train_X.shape, train_Y.shape)
        print("in data_validation", val_X.shape, val_Y.shape)
        print("in data_test", test_X.shape, test_Y.shape)

```

```

in data_train (8696, 41) (8696,) in data_validation (2175, 41) (2175,) in dat
a_test (1208, 41) (1208,)

```

Proper number of features

```

In [ ]: import matplotlib.pyplot as plt
        def combinelist(cd):
            num_features = cd['num_features']
            test_loss_lists = cd['test_loss']
            test_acc_lists = cd['test_acc']

            # Plot the test loss and accuracy for different numbers of features
            fig, ax1 = plt.subplots()

            color = 'tab:red'
            ax1.set_xlabel('Number of features')
            ax1.set_ylabel('validation loss', color=color)
            ax1.plot(num_features, test_loss_lists, color=color)
            ax1.tick_params(axis='y', labelcolor=color)

            ax2 = ax1.twinx() # instantiate a second axes that shares the same x-axis

            color = 'tab:blue'
            ax2.set_ylabel('validation accuracy', color=color) # we already handled the x-label
            ax2.plot(num_features, test_acc_lists, color=color)
            ax2.tick_params(axis='y', labelcolor=color)

            fig.tight_layout() # otherwise the right y-label is slightly clipped
            plt.show()

        # A simple model here
        # DNN(B)-frs(5240)-[12(relu)-8(relu)-1(sigmoid)]-[ep(8)]
        def simply_model(train_X, val_X):
            set_seed(5240)
            # epochs=8, batch_size=32

            # Create the DNN model using TensorFlow-Keras
            model = tf.keras.models.Sequential([
                tf.keras.layers.Dense(12, activation='relu', input_shape=(train_X.shape[1],)),
                tf.keras.layers.Dense(8, activation='relu'),
                tf.keras.layers.Dense(1, activation='sigmoid')
            ])

            # Compile the model
            model.compile(optimizer='adam',
                          loss='binary_crossentropy',
                          metrics=['accuracy'])

            # Train the model
            history = model.fit(
                train_X, train_Y,
                epochs=5, batch_size=32,
                validation_split=0.2,
                verbose=0)

```

```

# Training Performance
# val_loss, val_acc = history.history['val_loss'][-1], history.history['val_accuracy']

# Evaluate the model on the test set
test_loss, test_acc = model.evaluate(val_X, val_Y, verbose=0)

return test_loss, test_acc

def exam_modeling():
    get_X_Y_all()
    fs_len=len(features_list)
    fs = features_list['feature name'].to_list()

    # record
    s_h=pd.DataFrame(columns=['num_features', 'test_loss', 'test_acc'])

    for thre in range(5, fs_len+1, 2):
        newf = fs[:thre]

        # Use simple model when features vary
        test_loss, test_acc = simply_model(train_X[newf], val_X[newf])

        ht=[thre, test_loss, test_acc]
        s_h = mydic(ht, s_h)

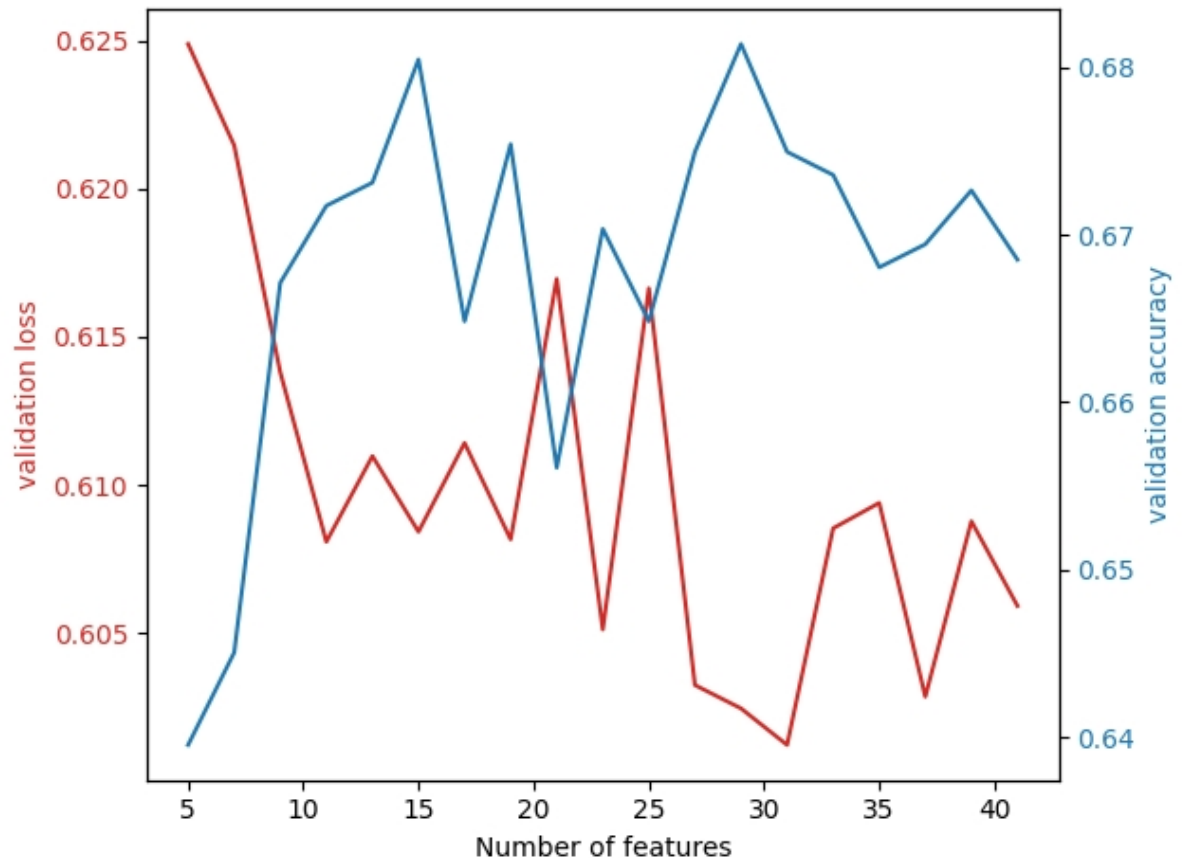
    return s_h

```

```
In [ ]: s_h = exam_modeling()
```

```
In [ ]: # obtain the best number of features
sh11 = s_h.sort_values('test_acc', ascending=False)
best_fs = sh11.head(1)['num_features'] # 29
# plot
print('Find the optimal number of features with the highest accuracy!\n', sh11.head(1))
combinelist(s_h)
```

```
Find the optimal number of features with the highest accuracy!      num_fea
tures test_loss test_acc12          29.0  0.602451  0.681379
```



Finding: Features, ranking TOP29 in the weight list, will be used in the following process.

DNN modeling

some functions

```
In [ ]: def generate_cd(num_features, test_loss_lists, test_acc_lists):
    cd = pd.DataFrame()
    cd['num_features'] = num_features
    cd['test_loss_lists'] = test_loss_lists
    cd['test_acc_lists'] = test_acc_lists
    return cd

def earllystopping_rst(history, early_stopping):
    es_rst = {}

    # Get the epoch at which training stopped
    es_rst['stopped_epoch'] = early_stopping.stopped_epoch

    # Get the history of the validation loss
    val_loss_history = history.history['val_loss']

    # Get the value of the validation loss at the last epoch
    es_rst['last_val_loss'] = val_loss_history[es_rst['stopped_epoch']]

    # Get the value of the validation loss at the best epoch
    es_rst['best_val_loss'] = min(val_loss_history)

    # Print the results
    # print("Training stopped at epoch: ", es_rst['stopped_epoch'])
    # print("Last validation loss: ", es_rst['last_val_loss'])
    # print("Best validation loss: ", es_rst['best_val_loss'])

    return es_rst
```

Based model

testing base model: # DNN(B)-frs(5240)-[12(relu)-8(relu)-1(sigmoid)]-[ep(30)]

Search based model

```
In [ ]: # create history doc
b_h = pd.DataFrame(columns=['layer', 'epochs', 'batch_size', 'val_loss', 'val_acc', 'test_lo
l2_h = pd.DataFrame(columns=['layer', 'epochs', 'batch_size', 'val_loss', 'val_acc', 'test_l
d_h = pd.DataFrame(columns=['layer', 'epochs', 'batch_size', 'val_loss', 'val_acc', 'test_lo
```

```
In [ ]: # Here are 3 modeling function
# Base model, L2-regularized model, Dropout model

# Base model-----
def m1(ly, epochs=30, batch_size=32, b_h=b_h):

    # set random seed
    set_seed(5240)
    # obtain X,Y
    get_X_Y_all_n(29)

    # Create the DNN model using TensorFlow-Keras
    model = tf.keras.models.Sequential()
    model.add(tf.keras.layers.Dense(ly[0][0], activation=ly[0][1], input_shape=(train_X.s
t=len(ly)
    for i in range(1,t):
        model.add(tf.keras.layers.Dense(ly[i][0], activation=ly[i][1]))

    # Compile the model
    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
```

```

# Define early stopping callback
early_stopping = EarlyStopping(monitor='val_loss', patience=20)

# Train the model
history = model.fit(
    train_X, train_Y,
    epochs=epochs, batch_size=batch_size,
    validation_data=(val_X, val_Y),
    callbacks=[early_stopping],
    verbose=0)

# early stopping
es_rst = earlystopping_rst(history, early_stopping)

# Training Performance
val_loss, val_acc = history.history['val_loss'][-1], history.history['val_accuracy'][-1]

# Evaluate the model on the test set
test_loss, test_acc = model.evaluate(test_X, test_Y, verbose=0)

print("Evaluation Result (test dataset):")
print(f'Test loss and Test accuracy: {test_loss:.3f}, {test_acc:.3f}')
print("*****")
# record
hist=[ly, epochs, batch_size, val_loss, val_acc, test_loss, test_acc]
b_h = mydic(hist, b_h)

return model, b_h

# L1-regularized model, regularizers.l1(0.001)-----
def m2(ly, epochs=30, batch_size=32, l2_h=l2_h):
    set_seed(5240)
    get_X_Y_all_n(29)

    # Create the DNN model using TensorFlow-Keras
    model = tf.keras.models.Sequential()

    model.add(tf.keras.layers.Dense(ly[0][0], activation=ly[0][1],
                                     kernel_regularizer=regularizers.l1(0.001),
                                     input_shape=(train_X.shape[1],)))

    t=len(ly)
    for i in range(1, t):
        model.add(tf.keras.layers.Dense(ly[i][0], activation=ly[i][1], kernel_regularizer=regularizers.l1(0.001)))

    # Compile the model
    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

    # Define early stopping callback
    early_stopping = EarlyStopping(monitor='val_loss', patience=20)

    # Train the model
    history = model.fit(
        train_X, train_Y,
        epochs=epochs, batch_size=32,
        validation_data=(val_X, val_Y),
        callbacks=[early_stopping],
        verbose=0)

    # early stopping
    es_rst = earlystopping_rst(history, early_stopping)

    # Training Performance
    val_loss, val_acc = history.history['val_loss'][-1], history.history['val_accuracy'][-1]

    # Evaluate the model on the test set
    test_loss, test_acc = model.evaluate(test_X, test_Y, verbose=0)

    print("Evaluation Result (test dataset):")
    print(f'Test loss and Test accuracy: {test_loss:.3f}, {test_acc:.3f}')
    print("*****")
    hist=[ly, epochs, batch_size, val_loss, val_acc, test_loss, test_acc]

```

```

l2_h = mydic(hist,l2_h)

return model,l2_h

# Dropout model, Dropout(0.1)-----
def m3(ly,epochs=30,batch_size=32,d_h=d_h):
    set_seed(5240)
    get_X_Y_all_n(29)

    # Create the DNN model using TensorFlow-Keras
    model = tf.keras.models.Sequential()

    model.add(tf.keras.layers.Dense(ly[0][0], activation=ly[0][1],
                                     input_shape=(train_X.shape[1],)))
    model.add(Dropout(0.1))

    t=len(ly)
    for i in range(1,t):
        model.add(tf.keras.layers.Dense(ly[i][0], activation=ly[i][1], kernel_regularizer=
        model.add(Dropout(0.1))

    # Compile the model
    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

    # Define early stopping callback
    early_stopping = EarlyStopping(monitor='val_loss', patience=20)

    # Train the model
    history = model.fit(
        train_X, train_Y,
        epochs=epochs, batch_size=32,
        validation_data=(val_X, val_Y),
        callbacks=[early_stopping],
        verbose=0)

    # early stopping
    es_rst = earlystopping_rst(history, early_stopping)

    # Training Performance
    val_loss,val_acc = history.history['val_loss'][-1], history.history['val_accuracy'][-1]

    # Evaluate the model on the test set
    test_loss, test_acc = model.evaluate(test_X, test_Y, verbose=0)

    print("Evaluation Result (test dataset):")
    print(f'Test loss and Test accuracy: {test_loss:.3f}, {test_acc:.3f}')
    print("*****")
    hist=[ly,epochs,batch_size,val_loss,val_acc,test_loss, test_acc]
    d_h = mydic(hist,d_h)

    return model,d_h

```

Experiment

small value of both layers and neurons

```

In [ ]: #DNN(B)-frs(5240)-[12(relu)-8(relu)-1(sigmoid)]-[ep(150)]
ly1=[[12,'relu'],[8,'relu'],[1,'sigmoid']]
m1l, b_h = m1(ly1,epochs=30,batch_size=40)
m22, l2_h = m2(ly1,epochs=30,batch_size=40)
m33, d_h = m3(ly1,epochs=30,batch_size=40)

```

```

Evaluation Result (test dataset):Test loss and Test accuracy: 0.575, 0.703*
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.611, 0.701*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.610, 0.702*****

```

```

In [ ]: c

```

Out[]: 40

In []: b_h

Out[]:

	layer	epochs	batch_size	val_loss	val_acc	test_loss	test_acc
0	[[12, relu], [8, relu], [1, sigmoid]]	30	40	0.588350	0.691954	0.575489	0.702815
1	[[20, relu], [12, relu], [8, relu], [1, sigmoid]]	30	40	0.566329	0.724598	0.568962	0.714404
2	[[20, relu], [8, relu], [12, relu], [1, sigmoid]]	30	40	0.575790	0.708506	0.565358	0.708609
3	[[30, relu], [12, relu], [24, relu], [12, relu]]	30	40	0.586152	0.720460	0.592949	0.695364
4	[[4, relu], [4, relu], [6, relu], [1, sigmoid]]	30	40	0.593751	0.690575	0.591289	0.681291
5	[[8, relu], [16, relu], [32, relu], [32, relu]]	30	40	0.585899	0.702529	0.596105	0.698676
6	[[85, relu], [64, relu], [32, relu], [1, sigmoid]]	30	40	0.795825	0.692874	0.739458	0.703642
7	[[120, relu], [85, relu], [64, relu], [32, relu]]	30	40	1.413192	0.684598	1.331259	0.685430
8	[[180, relu], [120, relu], [85, relu], [64, relu]]	30	40	1.457744	0.703448	1.445611	0.692881
9	[[240, relu], [120, relu], [85, relu], [64, relu]]	30	40	1.467441	0.702069	1.314986	0.702815
10	[[240, relu], [180, relu], [124, relu], [60, relu]]	30	40	1.381654	0.708506	1.377443	0.696192
11	[[240, relu], [180, relu], [150, relu], [80, relu]]	30	40	1.453334	0.696552	1.469629	0.702815
12	[[240, relu], [180, relu], [64, relu], [32, relu]]	30	40	1.564879	0.702529	1.369812	0.705298
13	[[280, relu], [240, relu], [180, relu], [64, relu]]	30	40	1.564893	0.714943	1.549793	0.702815
14	[[90, relu], [85, relu], [64, relu], [32, relu]]	30	20	1.287994	0.707126	1.233758	0.716887
15	[[90, relu], [85, relu], [64, relu], [32, relu]]	30	40	1.255192	0.684598	1.154019	0.690397
16	[[90, relu], [85, relu], [79, relu], [32, relu]]	30	20	1.188217	0.709425	1.090487	0.713576
17	[[90, relu], [85, relu], [79, relu], [32, relu]]	30	40	1.210024	0.691494	1.115583	0.710265
18	[[90, relu], [85, relu], [94, relu], [32, relu]]	30	20	1.125412	0.685977	1.023490	0.705298
19	[[90, relu], [85, relu], [94, relu], [32, relu]]	30	40	1.175708	0.691954	1.177159	0.672185
20	[[90, relu], [85, relu], [109, relu], [32, relu]]	30	20	1.185713	0.699770	1.153613	0.706126
21	[[90, relu], [85, relu], [109, relu], [32, relu]]	30	40	1.079814	0.692414	1.080952	0.699503
22	[[105, relu], [85, relu], [64, relu], [32, relu]]	30	20	1.159753	0.687816	1.106455	0.714404

	layer	epochs	batch_size	val_loss	val_acc	test_loss	test_acc
23	[[105, relu], [85, relu], [64, relu], [32, rel...	30	40	1.211582	0.693793	1.150484	0.712748
24	[[105, relu], [85, relu], [79, relu], [32, rel...	30	20	1.219814	0.683218	1.211942	0.687914
25	[[105, relu], [85, relu], [79, relu], [32, rel...	30	40	1.167652	0.702069	1.099587	0.711921
26	[[105, relu], [85, relu], [94, relu], [32, rel...	30	20	1.184069	0.700690	1.184304	0.700331
27	[[105, relu], [85, relu], [94, relu], [32, rel...	30	40	1.138856	0.692874	1.136785	0.701159
28	[[105, relu], [85, relu], [109, relu], [32, re...	30	20	1.275737	0.699770	1.347430	0.695364
29	[[105, relu], [85, relu], [109, relu], [32, re...	30	40	1.252820	0.685517	1.118970	0.687086
30	[[120, relu], [85, relu], [64, relu], [32, rel...	30	20	1.179058	0.702069	1.195734	0.712748
31	[[120, relu], [85, relu], [64, relu], [32, rel...	30	40	1.244872	0.696552	1.238294	0.694536
32	[[120, relu], [85, relu], [79, relu], [32, rel...	30	20	1.418924	0.706667	1.382767	0.701987
33	[[120, relu], [85, relu], [79, relu], [32, rel...	30	40	1.266178	0.702529	1.261491	0.682947
34	[[120, relu], [85, relu], [94, relu], [32, rel...	30	20	1.283110	0.700230	1.194345	0.724338
35	[[120, relu], [85, relu], [94, relu], [32, rel...	30	40	1.188399	0.702069	1.094080	0.713576
36	[[120, relu], [85, relu], [109, relu], [32, re...	30	20	1.320965	0.701609	1.226330	0.703642
37	[[120, relu], [85, relu], [109, relu], [32, re...	30	40	1.349165	0.692874	1.306393	0.688742
38	[[135, relu], [85, relu], [64, relu], [32, rel...	30	20	1.346543	0.693333	1.254020	0.701987
39	[[135, relu], [85, relu], [64, relu], [32, rel...	30	40	1.264681	0.714943	1.186901	0.730960
40	[[135, relu], [85, relu], [79, relu], [32, rel...	30	20	1.382081	0.698851	1.346322	0.695364
41	[[135, relu], [85, relu], [79, relu], [32, rel...	30	40	1.200604	0.687356	1.143217	0.700331
42	[[135, relu], [85, relu], [94, relu], [32, rel...	30	20	1.365646	0.696552	1.302509	0.706126
43	[[135, relu], [85, relu], [94, relu], [32, rel...	30	40	1.318243	0.698851	1.232308	0.709437
44	[[135, relu], [85, relu], [109, relu], [32, re...	30	20	1.338927	0.701149	1.365887	0.700331
45	[[135, relu], [85, relu], [109, relu], [32, re...	30	40	1.351892	0.719080	1.299808	0.712748

	layer	epochs	batch_size	val_loss	val_acc	test_loss	test_acc
46	[[90, relu], [85, relu], [109, relu], [32, relu]]	30	40	1.258811	0.684138	1.244079	0.698676
47	[[105, relu], [85, relu], [109, relu], [32, relu]]	30	40	1.350618	0.697011	1.142485	0.731788
48	[[120, relu], [85, relu], [109, relu], [32, relu]]	30	40	1.324526	0.697471	1.192584	0.702815
49	[[135, relu], [85, relu], [109, relu], [32, relu]]	30	40	1.322662	0.683218	1.254866	0.693709

```
In [ ]: ly1=[[20,'relu'],[12,'relu'],[8,'relu'],[1,'sigmoid']]
m11, b_h = m1(ly1,epochs=30,batch_size=40)
m22, l2_h = m2(ly1,epochs=30,batch_size=40)
m33, d_h = m3(ly1,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 0.569, 0.714*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.606, 0.709*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.616, 0.701*****

```
In [ ]: ly1=[[20,'relu'],[8,'relu'],[12,'relu'],[1,'sigmoid']]
m11, b_h = m1(ly1,epochs=30,batch_size=40)
m22, l2_h = m2(ly1,epochs=30,batch_size=40)
m33, d_h = m3(ly1,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 0.565, 0.709*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.600, 0.713*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.689, 0.577*****

```
In [ ]: ly1=[[30,'relu'],[12,'relu'],[24,'relu'],[12,'relu'],[8,'relu'],[1,'sigmoid']]
m11, b_h = m1(ly1,epochs=30,batch_size=40)
m22, l2_h = m2(ly1,epochs=30,batch_size=40)
m33, d_h = m3(ly1,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 0.593, 0.695*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.612, 0.720*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.689, 0.577*****

```
In [ ]: ly1=[[4,'relu'],[4,'relu'],[6,'relu'],[1,'sigmoid']]
m11, b_h = m1(ly1,epochs=30,batch_size=40)
m22, l2_h = m2(ly1,epochs=30,batch_size=40)
m33, d_h = m3(ly1,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 0.591, 0.681*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.595, 0.704*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.638, 0.677*****

```
In [ ]: ly1=[[8,'relu'],[16,'relu'],[32,'relu'],[32,'relu'],[64,'relu'],[1,'sigmoid']]
m11, b_h = m1(ly1,epochs=30,batch_size=40)
m22, l2_h = m2(ly1,epochs=30,batch_size=40)
m33, d_h = m3(ly1,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 0.596, 0.699*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.615, 0.707*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.689, 0.577*****

larger value of both layers and neurons

```
In [ ]: #DNN(B)-frs(5240)-[85(relu)-64(relu)-32(relu)-1(sigmoid)]-[ep(150)-bs(10)]
lyl=[[85,'relu'],[64,'relu'],[32,'relu'],[1,'sigmoid']]
m1l, b_h = m1(lyl,epochs=30,batch_size=40)
m2l, l2_h = m2(lyl,epochs=30,batch_size=40)
m3l, d_h = m3(lyl,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 0.739, 0.704*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.608, 0.713*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.609, 0.700*****

```
In [ ]: lyl=[[120,'relu'],[85,'relu'],[64,'relu'],[32,'relu'],[1,'sigmoid']]
m1l, b_h = m1(lyl,epochs=30,batch_size=40)
m2l, l2_h = m2(lyl,epochs=30,batch_size=40)
m3l, d_h = m3(lyl,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 1.331, 0.685*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.612, 0.709*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.611, 0.718*****

```
In [ ]: lyl=[[180,'relu'],[120,'relu'],[85,'relu'],[64,'relu'],[32,'relu'],[1,'sigmoid']]
m1l, b_h = m1(lyl,epochs=30,batch_size=40)
m2l, l2_h = m2(lyl,epochs=30,batch_size=40)
m3l, d_h = m3(lyl,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 1.446, 0.693*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.621, 0.696*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.627, 0.719*****

```
In [ ]: lyl=[[240,'relu'],[120,'relu'],[85,'relu'],[64,'relu'],[32,'relu'],[1,'sigmoid']]
m1l, b_h = m1(lyl,epochs=30,batch_size=40)
m2l, l2_h = m2(lyl,epochs=30,batch_size=40)
m3l, d_h = m3(lyl,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 1.315, 0.703*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.627, 0.690*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.625, 0.714*****

```
In [ ]: lyl=[[240,'relu'],[180,'relu'],[124,'relu'],[60,'relu'],[32,'relu'],[1,'sigmoid']]
m1l, b_h = m1(lyl,epochs=30,batch_size=40)
m2l, l2_h = m2(lyl,epochs=30,batch_size=40)
m3l, d_h = m3(lyl,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 1.377, 0.696*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.614, 0.708*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.616, 0.724*****

```
In [ ]: lyl=[[240,'relu'],[180,'relu'],[150,'relu'],[80,'relu'],[32,'relu'],[1,'sigmoid']]
m1l, b_h = m1(lyl,epochs=30,batch_size=40)
m2l, l2_h = m2(lyl,epochs=30,batch_size=40)
m3l, d_h = m3(lyl,epochs=30,batch_size=40)
```

Evaluation Result (test dataset):Test loss and Test accuracy: 1.470, 0.703*
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.636, 0.692*****
 *****Evaluation Result (test dataset):Test loss and Test accuracy: 0.618, 0.716*****

```
In [ ]: #DNN(B)-frs(5240)-[([240,'relu']-[180,'relu']-[64,'relu']-[32,'relu']-[24,'relu']-1(sigmoid))
lyl=[[240,'relu'],[180,'relu'],[64,'relu'],[32,'relu'],[24,'relu'],[1,'sigmoid']]
m1l, b_h = m1(lyl,epochs=30,batch_size=c)
m2l, l2_h = m2(lyl,epochs=30,batch_size=c)
m3l, d_h = m3(lyl,epochs=30,batch_size=c)
```

```
Evaluation Result (test dataset):Test loss and Test accuracy: 1.370, 0.705*
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.631, 0.697*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.624, 0.709*****
```

```
In [ ]: #DNN(B)-frs(5240)-[[240,'relu']-[180,'relu']-[64,'relu']-[32,'relu']-[24,'relu']-1(sigmoid)]
lyl=[[280,'relu'],[240,'relu'],[180,'relu'],[64,'relu'],[32,'relu'],[24,'relu'],[1,'sigmoid']]
m1l, b_h = m1(lyl,epochs=30,batch_size=40)
m22, l2_h = m2(lyl,epochs=30,batch_size=40)
m33, d_h = m3(lyl,epochs=30,batch_size=40)
```

```
Evaluation Result (test dataset):Test loss and Test accuracy: 1.550, 0.703*
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.698, 0.577*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.689, 0.577*****
```

```
In [ ]: ly1=[[64,'relu']-[85,'relu']-[120,'relu']-[32,'relu'],[1,'sigmoid']]
m1l, b_h = m1(ly1,epochs=30,batch_size=40)
m22, l2_h = m2(ly1,epochs=30,batch_size=40)
m33, d_h = m3(ly1,epochs=30,batch_size=40)
```

Fine tuning

DNN(B)-frs(5240)-([120,'relu']-[85,'relu']-[64,'relu']-[32,'relu']-1(sigmoid))-[ep(30)]

```
In [ ]: # DNN(B)-frs(5240)-[[120,'relu']-[85,'relu']-[64,'relu']-[32,'relu']-1(sigmoid)]-[ep(30)]
for a in range(90,150,15):
    for b in range(64,120,15):
        for c in [20,40]:
            print('-----')
            print(a,b,c)
            ly1=[[a,'relu'],[85,'relu'],[b,'relu'],[32,'relu'],[1,'sigmoid']]
            m1l, b_h = m1(ly1,epochs=30,batch_size=c)
            m22, l2_h = m2(ly1,epochs=30,batch_size=c)
            m33, d_h = m3(ly1,epochs=30,batch_size=c)
```

-----90 64 20Evaluation Result (test dataset):Test loss
and Test accuracy: 1.234, 0.717*****Evaluation Result (test d
ataset):Test loss and Test accuracy: 0.609, 0.703*****Evaluati
on Result (test dataset):Test loss and Test accuracy: 0.610, 0.719*****
*****-----90 64 40Evaluation Result (test dataset):
Test loss and Test accuracy: 1.154, 0.690*****Evaluation Result
(test dataset):Test loss and Test accuracy: 0.612, 0.704*****
*Evaluation Result (test dataset):Test loss and Test accuracy: 0.618, 0.712
*****-----90 79 20Evaluation Result (test
dataset):Test loss and Test accuracy: 1.090, 0.714*****Evaluat
ion Result (test dataset):Test loss and Test accuracy: 0.600, 0.715*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.6
12, 0.720*****-----90 79 40Evaluation Resu
lt (test dataset):Test loss and Test accuracy: 1.116, 0.710*****
**Evaluation Result (test dataset):Test loss and Test accuracy: 0.613, 0.70
4*****Evaluation Result (test dataset):Test loss and Test accu
racy: 0.614, 0.714*****-----90 94 20Evalua
tion Result (test dataset):Test loss and Test accuracy: 1.023, 0.705*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.
616, 0.697*****Evaluation Result (test dataset):Test loss and
Test accuracy: 0.612, 0.722*****-----90 94
40Evaluation Result (test dataset):Test loss and Test accuracy: 1.177, 0.6
72*****Evaluation Result (test dataset):Test loss and Test acc
uracy: 0.614, 0.702*****Evaluation Result (test dataset):Test
loss and Test accuracy: 0.610, 0.718*****-----
----90 109 20Evaluation Result (test dataset):Test loss and Test accuracy:
1.154, 0.706*****Evaluation Result (test dataset):Test loss an
d Test accuracy: 0.630, 0.698*****Evaluation Result (test data
set):Test loss and Test accuracy: 0.611, 0.719*****-----
-----90 109 40Evaluation Result (test dataset):Test loss and Test
accuracy: 1.081, 0.700*****Evaluation Result (test dataset):Te
st loss and Test accuracy: 0.612, 0.710*****Evaluation Result
(test dataset):Test loss and Test accuracy: 0.613, 0.720*****--
-----105 64 20Evaluation Result (test dataset):Test loss
and Test accuracy: 1.106, 0.714*****Evaluation Result (test d
ataset):Test loss and Test accuracy: 0.615, 0.714*****Evaluati
on Result (test dataset):Test loss and Test accuracy: 0.611, 0.719*****
*****-----105 64 40Evaluation Result (test dataset)
:Test loss and Test accuracy: 1.150, 0.713*****Evaluation Result
(test dataset):Test loss and Test accuracy: 0.611, 0.698*****
**Evaluation Result (test dataset):Test loss and Test accuracy: 0.609, 0.71
9*****-----105 79 20Evaluation Result (tes
t dataset):Test loss and Test accuracy: 1.212, 0.688*****Evalu
ation Result (test dataset):Test loss and Test accuracy: 0.606, 0.718*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0
.616, 0.709*****-----105 79 40Evaluation R
esult (test dataset):Test loss and Test accuracy: 1.100, 0.712*****
****Evaluation Result (test dataset):Test loss and Test accuracy: 0.617, 0
.695*****Evaluation Result (test dataset):Test loss and Test a
ccuracy: 0.615, 0.719*****-----105 94 20Ev
aluation Result (test dataset):Test loss and Test accuracy: 1.184, 0.700***
*****Evaluation Result (test dataset):Test loss and Test accuracy
: 0.614, 0.695*****Evaluation Result (test dataset):Test loss
and Test accuracy: 0.608, 0.712*****-----1
05 94 40Evaluation Result (test dataset):Test loss and Test accuracy: 1.137
, 0.701*****Evaluation Result (test dataset):Test loss and Tes
t accuracy: 0.613, 0.709*****Evaluation Result (test dataset):
Test loss and Test accuracy: 0.612, 0.717*****-----
-----105 109 20Evaluation Result (test dataset):Test loss and Test accu
racy: 1.347, 0.695*****Evaluation Result (test dataset):Test l
oss and Test accuracy: 0.611, 0.700*****Evaluation Result (tes
t dataset):Test loss and Test accuracy: 0.613, 0.719*****-----
-----105 109 40Evaluation Result (test dataset):Test loss an
d Test accuracy: 1.119, 0.687*****Evaluation Result (test data

set):Test loss and Test accuracy: 0.615, 0.704*****Evaluation
Result (test dataset):Test loss and Test accuracy: 0.612, 0.722*****
*****-----120 64 20Evaluation Result (test dataset):Te
st loss and Test accuracy: 1.196, 0.713*****Evaluation Result
(test dataset):Test loss and Test accuracy: 0.611, 0.704*****E
valuation Result (test dataset):Test loss and Test accuracy: 0.610, 0.722**
*****-----120 64 40Evaluation Result (test d
ataset):Test loss and Test accuracy: 1.238, 0.695*****Evaluati
on Result (test dataset):Test loss and Test accuracy: 0.610, 0.709*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.61
4, 0.714*****-----120 79 20Evaluation Resu
lt (test dataset):Test loss and Test accuracy: 1.383, 0.702*****
**Evaluation Result (test dataset):Test loss and Test accuracy: 0.618, 0.69
5*****Evaluation Result (test dataset):Test loss and Test accu
racy: 0.609, 0.724*****-----120 79 40Evalu
ation Result (test dataset):Test loss and Test accuracy: 1.261, 0.683*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0
.612, 0.701*****Evaluation Result (test dataset):Test loss and
Test accuracy: 0.612, 0.715*****-----120
94 20Evaluation Result (test dataset):Test loss and Test accuracy: 1.194, 0
.724*****Evaluation Result (test dataset):Test loss and Test a
ccuracy: 0.612, 0.714*****Evaluation Result (test dataset):Tes
t loss and Test accuracy: 0.611, 0.728*****-----
-----120 94 40Evaluation Result (test dataset):Test loss and Test accuracy
: 1.094, 0.714*****Evaluation Result (test dataset):Test loss
and Test accuracy: 0.619, 0.702*****Evaluation Result (test da
taset):Test loss and Test accuracy: 0.612, 0.721*****-----
-----120 109 20Evaluation Result (test dataset):Test loss and Te
st accuracy: 1.226, 0.704*****Evaluation Result (test dataset)
:Test loss and Test accuracy: 0.613, 0.698*****Evaluation Resu
lt (test dataset):Test loss and Test accuracy: 0.613, 0.711*****
**-----120 109 40Evaluation Result (test dataset):Test
loss and Test accuracy: 1.306, 0.689*****Evaluation Result (te
st dataset):Test loss and Test accuracy: 0.613, 0.704*****Eval
uation Result (test dataset):Test loss and Test accuracy: 0.610, 0.724****
*****-----135 64 20Evaluation Result (test data
set):Test loss and Test accuracy: 1.254, 0.702*****Evaluation
Result (test dataset):Test loss and Test accuracy: 0.618, 0.697*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.615,
0.714*****-----135 64 40Evaluation Result
(test dataset):Test loss and Test accuracy: 1.187, 0.731*****E
valuation Result (test dataset):Test loss and Test accuracy: 0.618, 0.701**
*****Evaluation Result (test dataset):Test loss and Test accurac
y: 0.608, 0.719*****-----135 79 20Evaluati
on Result (test dataset):Test loss and Test accuracy: 1.346, 0.695*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.61
1, 0.704*****Evaluation Result (test dataset):Test loss and Te
st accuracy: 0.609, 0.724*****-----135 79
40Evaluation Result (test dataset):Test loss and Test accuracy: 1.143, 0.70
0*****Evaluation Result (test dataset):Test loss and Test accu
racy: 0.607, 0.715*****Evaluation Result (test dataset):Test l
oss and Test accuracy: 0.616, 0.718*****-----
---135 94 20Evaluation Result (test dataset):Test loss and Test accuracy: 1
.303, 0.706*****Evaluation Result (test dataset):Test loss and
Test accuracy: 0.612, 0.710*****Evaluation Result (test datas
et):Test loss and Test accuracy: 0.609, 0.726*****-----
-----135 94 40Evaluation Result (test dataset):Test loss and Test a
ccuracy: 1.232, 0.709*****Evaluation Result (test dataset):Tes
t loss and Test accuracy: 0.615, 0.691*****Evaluation Result (t
est dataset):Test loss and Test accuracy: 0.612, 0.730*****---
-----135 109 20Evaluation Result (test dataset):Test loss
and Test accuracy: 1.366, 0.700*****Evaluation Result (test d
ataset):Test loss and Test accuracy: 0.615, 0.699*****Evaluati
on Result (test dataset):Test loss and Test accuracy: 0.613, 0.722*****

```
*****-----135 109 40Evaluation Result (test dataset
):Test loss and Test accuracy: 1.300, 0.713*****Evaluation Res
ult (test dataset):Test loss and Test accuracy: 0.616, 0.700*****
***Evaluation Result (test dataset):Test loss and Test accuracy: 0.612, 0.7
19*****
```

```
In [ ]: for a in [120,135]:
        for b in [79,94]:
            for e in [30,80,120,160,250]:
                print('-----')
                print(a,b,e)
                ly1=[a,'relu'],[85,'relu'],[b,'relu'],[32,'relu'],[1,'sigmoid']]
                m1l, b_h = m1(ly1,epochs=30,batch_size=40)
                m22, l2_h = m2(ly1,epochs=30,batch_size=40)
                m33, d_h = m3(ly1,epochs=30,batch_size=40)
```

-----120 79 30Evaluation Result (test dataset):Test loss and Test accuracy: 1.220, 0.685*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.613, 0.700*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.608, 0.728*****
-----120 79 80Evaluation Result (test dataset):Test loss and Test accuracy: 1.105, 0.709*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.612, 0.713*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.618, 0.711**
-----120 79 120Evaluation Result (test dataset):Test loss and Test accuracy: 1.115, 0.690*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.622, 0.709*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.613, 0.719*****
-----120 79 160Evaluation Result (test dataset):Test loss and Test accuracy: 1.237, 0.695*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.608, 0.719*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.611, 0.721*****
-----120 79 250Evaluation Result (test dataset):Test loss and Test accuracy: 1.077, 0.712*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.615, 0.706*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.609, 0.720*****
-----120 94 30Evaluation Result (test dataset):Test loss and Test accuracy: 1.139, 0.704*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.616, 0.698*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.612, 0.722*****
-----120 94 80Evaluation Result (test dataset):Test loss and Test accuracy: 1.249, 0.692*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.619, 0.700*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.613, 0.719*****
-----120 94 120Evaluation Result (test dataset):Test loss and Test accuracy: 1.271, 0.686*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.621, 0.690*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.612, 0.729*****

-----120 94 160Evaluation Result (test dataset):Test loss and Test accuracy: 1.314, 0.692*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.608, 0.717*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.612, 0.722**
-----120 94 250Evaluation Result (test dataset):Test loss and Test accuracy: 1.222, 0.699*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.608, 0.711*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.613, 0.711*****
-----135 79 30Evaluation Result (test dataset):Test loss and Test accuracy: 1.213, 0.698*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.611, 0.710*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.616, 0.714*****
-----135 79 80Evaluation Result (test dataset):Test loss and Test accuracy: 1.036, 0.715*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.626, 0.702*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.611, 0.717*****
-----135 79 120Evaluation Result (test dataset):Test loss and Test accuracy: 1.323, 0.700*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.615, 0.706*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.610, 0.719*****
-----135 79 160Evaluation Result (test dataset):Test loss and Test accuracy: 1.292, 0.709*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.619, 0.697*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.611, 0.722*****
-----135 79 250Evaluation Result (test dataset):Test loss and Test accuracy: 1.299, 0.704*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.622, 0.692*****
Evaluation Result (test dataset):Test loss and Test accuracy: 0.611, 0.728*****

-----135 94 30Evaluation Result (test dataset):Test loss and Test accuracy: 1.128, 0.708*****Evaluation Result (test dataset):Test loss and Test accuracy: 1.128, 0.708*****Evaluation Result (test dataset):Test loss and Test accuracy: 1.128, 0.708*****


```

lt (test dataset):Test loss and Test accuracy: 0.627, 0.694*****
**Evaluation Result (test dataset):Test loss and Test accuracy: 0.613, 0.72
2*****-----135 94 80Evaluation Result (tes
t dataset):Test loss and Test accuracy: 1.284, 0.692*****Evaluation Resu
lt (test dataset):Test loss and Test accuracy: 0.619, 0.704*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0
.610, 0.719*****-----135 94 120Evaluation
Result (test dataset):Test loss and Test accuracy: 1.331, 0.681*****
*****Evaluation Result (test dataset):Test loss and Test accuracy: 0.617,
0.697*****Evaluation Result (test dataset):Test loss and Test
accuracy: 0.609, 0.729*****-----135 94 160
Evaluation Result (test dataset):Test loss and Test accuracy: 1.300, 0.709*
*****Evaluation Result (test dataset):Test loss and Test accura
cy: 0.614, 0.709*****Evaluation Result (test dataset):Test los
s and Test accuracy: 0.608, 0.738*****-----
-135 94 250Evaluation Result (test dataset):Test loss and Test accuracy: 1.
199, 0.696*****Evaluation Result (test dataset):Test loss and
Test accuracy: 0.614, 0.697*****Evaluation Result (test datase
t):Test loss and Test accuracy: 0.611, 0.718*****

```

```

In [ ]: # DNN(B)-frs(5240)-[([120,'relu']-[85,'relu']-[64,'relu']-[32,'relu']-1(sigmoid))-[ep(3
for a in range(90,150,15):
    print('-----')
    print(a)
    ly1=[[a,'relu'],[85,'relu'],[b,'relu'],[32,'relu'],[1,'sigmoid']]
    m1l, b_h = m1(ly1,epochs=30,batch_size=40)
    m2l, l2_h = m2(ly1,epochs=30,batch_size=40)
    m3l, d_h = m3(ly1,epochs=30,batch_size=40)

```

```

-----90Evaluation Result (test dataset):Test loss and T
est accuracy: 1.244, 0.699*****Evaluation Result (test datase
t):Test loss and Test accuracy: 0.623, 0.696*****Evaluation Resu
lt (test dataset):Test loss and Test accuracy: 0.610, 0.724*****
***-----105Evaluation Result (test dataset):Test loss a
nd Test accuracy: 1.142, 0.732*****Evaluation Result (test dat
aset):Test loss and Test accuracy: 0.614, 0.708*****Evaluation
Result (test dataset):Test loss and Test accuracy: 0.614, 0.716*****
*****-----120Evaluation Result (test dataset):Test lo
ss and Test accuracy: 1.193, 0.703*****Evaluation Result (test
dataset):Test loss and Test accuracy: 0.616, 0.698*****Evalua
tion Result (test dataset):Test loss and Test accuracy: 0.610, 0.719*****
*****-----135Evaluation Result (test dataset):Tes
t loss and Test accuracy: 1.255, 0.694*****Evaluation Result (
test dataset):Test loss and Test accuracy: 0.605, 0.715*****Ev
aluation Result (test dataset):Test loss and Test accuracy: 0.610, 0.724***
*****

```

Export excel file

Tips: correct the name of file l2_h

```

In [ ]: l1_h=l2_h

```

```

In [ ]: b_h.sort_values(by=['test_acc']).head(2)

```

```

Out[ ]:

```

	layer	epochs	batch_size	val_loss	val_acc	test_loss	test_acc
19	[[90, relu], [85, relu], [94, relu], [32, relu]...	30	40	1.175708	0.691954	1.177159	0.672185
4	[[4, relu], [4, relu], [6, relu], [1, sigmoid]]	30	40	0.593751	0.690575	0.591289	0.681291

```
In [ ]: l1_h.sort_values(by=['test_acc']).head(2)
```

```
Out[ ]:
```

	layer	epochs	batch_size	val_loss	val_acc	test_loss	test_acc
13	[[280, relu], [240, relu], [180, relu], [64, r...	30	40	0.703460	0.559080	0.698405	0.576987
9	[[240, relu], [120, relu], [85, relu], [64, re...	30	40	0.624535	0.696552	0.627268	0.690397

```
In [ ]: d_h.sort_values(by=['test_acc']).head(2)
```

```
Out[ ]:
```

	layer	epochs	batch_size	val_loss	val_acc	test_loss	test_acc
2	[[20, relu], [8, relu], [12, relu], [1, sigmoid]]	30	40	0.690158	0.55908	0.689127	0.576987
3	[[30, relu], [12, relu], [24, relu], [12, relu...	30	40	0.690229	0.55908	0.689226	0.576987

```
In [ ]: b_h.to_excel('base_model_recording.xlsx')
```

```
In [ ]: l1_h.to_excel('L1_regularized_model_recording.xlsx')
```

```
In [ ]: d_h.to_excel('Dropout_model_recording.xlsx')
```